Inside the Spec

Real Scenarios. Working Artifacts.
Decisions That Matter.

The API timed out. Did the payment succeed?

One top-up. An unknown outcome.
A specification for what happens next.

CASE 01 · PAYMENT TIMEOUT

€50. No reliable answer.

A user tops up a balance. The provider may accept the payment before our request times out.

01 **The request times out**
We cannot tell whether a payment exists.

02 **The user tries again**
Reuse the existing intent and client key.

03 **Evidence arrives later**
Verify the result before changing the payment.

> **A timeout does not prove that the payment failed.**

Case assumptions: one provider; a 24-hour idempotency window, including concurrent calls; lookup by a stable reference; immutable final outcomes. Verify these for a real integration.

Three independent statuses.

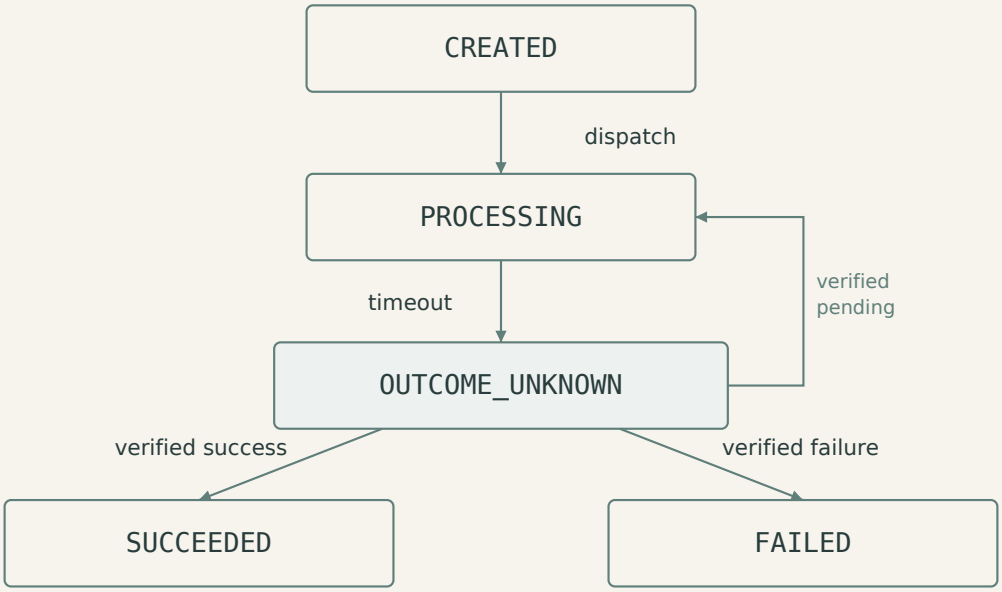
payment_status	SUCCEEDED	The provider confirmed payment success.
credit_status	PENDING	The balance posting is not confirmed yet.
review_status	OPEN	At least one review reason remains open.

These values can coexist. One combined “payment status” would hide which part still needs work.

A credit failure does not make the payment FAILED. A review flag does not replace the known payment outcome.

Keep the unknown visible.

The diagram follows recovery after an ambiguous create request. State names describe what our service knows.

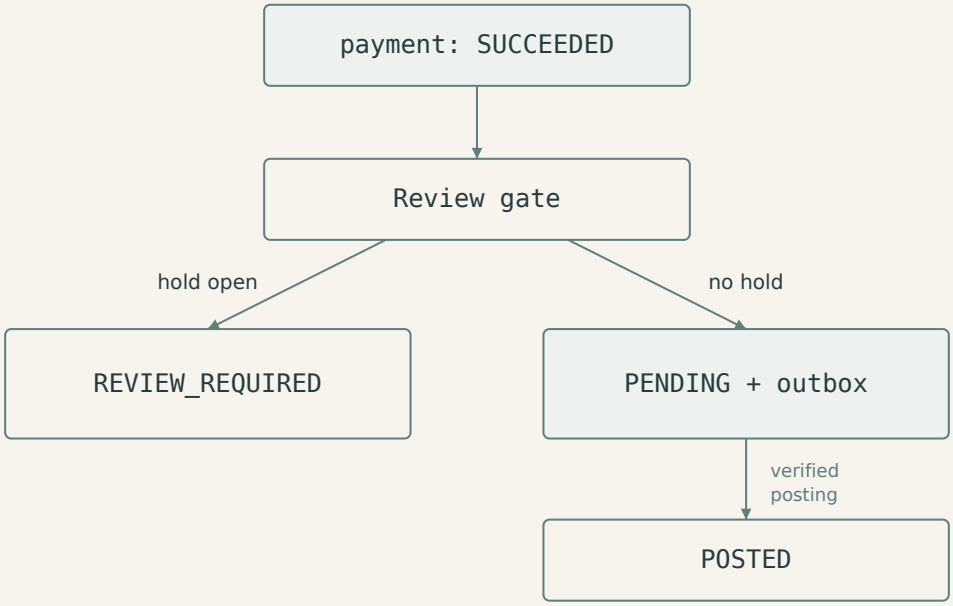


Also allowed: PROCESSING → either verified final outcome, and CREATED → FAILED for a definitive local rejection before dispatch. Full transition rules are in the specification.

> **Neither NOT_FOUND nor an expired window means FAILED.**

Link success to credit.

Payment success, review checks, and the credit task have an explicit connection.



The payment DB commits success and any outbox task locally. A later balance-DB transaction creates the posting. CR-01/CR-02 define the gate and dispatch boundary.

One intent. Stable identity.

Identifier	What it identifies
client_idempotency_key	One user intent. Retain it across reloads and network failures.
payment_id	One internal payment operation.
provider_idempotency_key	The same provider operation across create retries.
attempt_id	One outgoing CREATE or LOOKUP call.

> **Same client key + same parameters → return the existing operation.**

Key scope: user + operation type + client key. The fingerprint covers normalized amount, currency, and instrument.

Save the key, fingerprint, and operation atomically. A storage constraint enforces uniqueness.

Specify the repeated request.

Condition	API behavior
Same key, same data; any nonfinal state	202 · Full PaymentStatus DTO. Continue with GET.
Same key, changed data	409 · IDEMPOTENCY_KEY_REUSED.
Same key, final payment	200 · Same outcome; report credit state separately.
New key; matching unresolved payment	409 · Existing PaymentStatus + SHOW_EXISTING_PAYMENT.
Cannot save operation reliably	503 · No external call without confirmed durable storage.

> **A client retry does not itself trigger another provider create.**

If a commit response is lost, a retry with the same key finds the operation if it was saved. Do not switch keys to recover.

202 returns a status object.

POST uses 202 for all three nonfinal states. This is a DTO fragment; the full schema also includes credit_status and review_status.

```
payment_id: pay-001
payment_status: PROCESSING
status_url: /payments/pay-001
next_action: WAIT
poll_after_seconds: 5
retry_post: false
```

> **On 202: stop POST retries. Read the same operation with GET.**

GET returns 200 even for unknown outcomes. Poll at most once per 5 seconds, for at most 60 seconds, then stop and retain a refresh action.

A review returns CONTACT_SUPPORT. A duplicate-key guard response returns the complete existing operation and SHOW_EXISTING_PAYMENT. HTTP 200 alone does not mean the payment succeeded.

Bound the recovery loop.

```
CREATE / LOOKUP timeout: 10 seconds
LOOKUP: +15s, +1m, +5m, +15m, +1h
CREATE budget: initial + 1 retry
```

For an unknown outcome, lookup comes first. NOT_FOUND permits a retry only if the original provider key, parameters, budget, and safe window are still valid.

> **No result at 1 hour → manual review. Stop automatic create; continue accepting late evidence.**

Provider window: 24 hours. Local cutoff: before first dispatch +23 hours. The earlier one-hour review limit also applies.

Case choices. Disable create retries if safe window semantics cannot be verified. After one hour: owned review plus daily reconciliation; neither creates a new payment.

An HTTP code is not proof.

Provider response	Recovery decision
429; proven nonacceptance	Honor Retry-After. Retry only if earlier attempts are also proven not accepted.
429; no nonacceptance guarantee	Outcome unknown. Lookup before an eligible retry.
502 / 503 / 504 or timeout	Unknown without a separate contractual guarantee.
Documented definitive rejection	FAILED only when the contract proves final failure.

> **Every create call uses the same total budget, including 429 responses.**

Proven nonacceptance of attempt 2 does not resolve an unknown attempt 1. Never infer the payment result from a failed lookup.

Close unknown with evidence.

Evidence	Decision
Verified final success / failure	Apply normal final transition; credit gate still applies.
Missing row / NOT_FOUND	Remain unknown. Keep guard and owner.
Final NOT_EXECUTED under PA-08	FAILED only if all attempts are covered and future execution is fenced.

> **Daily reconciliation: 02:00 UTC. Escalate at 24 hours; incident at 7 days. Age never creates a financial result.**

PA-08 is an additional case requirement, not an assumed real-provider feature. If final nonexecution cannot be proven, do not mark FAILED just to clear the queue.

A new key is still a repeat risk.

Block a new key while a matching payment is unresolved, regardless of age.

> **Same user + instrument + amount + currency, with CREATED, PROCESSING, or OUTCOME_UNKNOWN → 409.**

Return the original operation’s full status and SHOW_EXISTING_PAYMENT. Check the existing key first; serialize guard, creation, and resolution.

At 6 minutes	At 24 hours
Still unknown → still blocked.	Still unknown → still blocked.

No timed expiry or user override. Different instruments/amounts and new intents after final success need separate policy. Waiting for evidence reduces availability by design.

Review has reasons, not one flag.

review_status aggregates separate review items. Close one reason without erasing the others.

Late verified success arrives	Expected behavior
Only RESULT_OVERDUE is open	Set SUCCEEDED; close overdue; schedule credit.
Overdue + SECURITY_HOLD	Set SUCCEEDED; close only overdue; hold credit.
A final failure already exists	Keep the recorded final state; open a conflict review.

> **An operator cannot assign success without provider evidence.**

Review aggregate: NONE → OPEN → RESOLVED; new evidence may reopen it. Every item has an owner, next action, evidence, and an audit trail.

Verify. Persist. Then acknowledge.

A webhook is a provider notification. This illustrative contract verifies the original body before applying business data.

```
HMAC-SHA256(timestamp + "." + raw_body)
Delivery timestamp: within ±300 seconds
Deduplication: provider + event_id
```

Keep secrets in a secret manager. Check the merchant, reference, provider payment ID, amount, and currency. Compare decoded signatures in constant time.

> **Return 2xx only after durable inbox storage. If storage fails, return 503.**

event_id is mandatory in the signed body. Missing ID → 400; no business update. Replays hit the unique inbox key, even within 300 seconds. A new delivery timestamp does not create a new event.

Success still needs a posting.

01 **Commit payment success**

In the same transaction, create the credit task if no blocking review exists.

02 **Recheck before dispatch**

An open hold pauses an unstarted task. Removing the final hold enables delivery.

03 **Make redelivery safe**

The balance service atomically enforces one posting per payment_id.

> **Balance timeout or 5xx → retry the same credit task, never create another payment.**

Use backoff with jitter, shared rate limits, and a circuit breaker. Retain undelivered tasks; after 24 hours, move them to manual recovery.

A hold cannot revoke a call already started. Case SLO: 99.9% of eligible credits within 60 seconds; alert after 2 minutes. The full specification defines the denominator and exclusions. This target has not been measured.

A queue read is not authorization.

Before dispatch, lock the payment row, reread holds, and commit DISPATCH_AUTHORIZED. Hold writers lock the same row.

Commit order	Consequence
Hold before authorization	No balance call, even if the worker already read the task.
Authorization before hold	That attempt may execute. Hold stops later attempts.
First response lost, no hold	Redeliver same payment_id; return or create one posting.

> **Outbox atomicity is local. The balance DB commits separately and enforces posting uniqueness.**

Authorization commit is the logical start boundary. A later hold cannot strictly revoke this call. Strict revocation needs a balance-owned protocol, not another local read.

The second writer must reread.

Concurrent evidence	Required result
Lookup PENDING + success webhook	SUCCEEDED; old pending cannot downgrade it.
Success from both channels	One transition, one logical credit task, one posting.
Final success + final failure	Retain first accepted final state; open conflict.
Timeout + verified success	Retain success regardless of commit order.

Persist evidence. Check the operation version. Commit the transition and outbox atomically. On conflict, reload and reevaluate the same evidence.

Local locks cannot cancel an in-flight request. Overlapping provider calls also require the provider’s concurrent idempotency guarantee.

Test the uncomfortable cases.

Scenario	Acceptance result
AC-03 · Both create calls time out	Unknown remains; no third create.
AC-09 · Late success during security hold	Success recorded; only overdue closes; credit held.
AC-18 · Posting exists, response lost	Redelivery returns the same posting.
AC-22 · Different keys arrive together	Guard accepts one operation; the other gets 409.
AC-23 · New key after 6 minutes	409 with existing operation; no second payment.

> **The full specification contains 40 acceptance scenarios linked to the rules.**

These are expected outcomes, not executed implementation tests. Provider assumptions, concurrency, and the SLO still require validation against a real system.

Beyond the Scope

Process	What this case does not specify
3DS; authorization / capture	User action, authorization, and capture lifecycles.
Refund; reversal; chargeback	Separate financial operations after payment success.
Provider failover; partials; FX	Cross-provider duplicates, remaining amounts, and conversion.
Provider report adapters	Report formats and evidence of finality.

Also separate: correcting final outcomes, strict balance-side revocation, and duplicates across instruments or after final success.

Full specification – see GitHub.

GitHub Repository for the Inside the Spec

Yulia Melekhova on LinkedIn